LAPORAN TUGAS BESAR
SISTEM MANAJEMEN PEMINJAMAN RUANG KELAS BERBASIS OBJEK "CLASSMATE"

Diajukan untuk memenuhi sebagian dari syarat
Kelulusan mata kuliah Pemrograman Berorientasi Objek

Dosen Pengampu: Nurul Ilmi, S.Kom., M.T.



Universitas
Telkom

KELOMPOK 7

Fazli Radika	103062300092
Muhammad Afriza Hidayat	103062300093
Nomensen Melkisedek Pardosi	103062330101

S1IT-KJ-23-002

PROGRAM STUDI S1 TEKNOLOGI INFORMASI
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY
JAKARTA

2025

BAB I **3**
PENDAHULUAN **3**
1.1 Latar Belakang 3
1.2 Rumusan Masalah 3
1.3 Batasan Masalah 3
1.4 Tujuan 3
1.5 Manfaat 4
BAB II **5**
ANALISIS DAN PERANCANGAN SISTEM **5**
2.1 Deskripsi Umum Sistem 5
2.2 Daftar Fitur Utama 5
2.2.1 FITUR 1: AUTENTIKASI & MANAJEMEN USER 5
2.2.2 FITUR 2: BOOKING & PEMINJAMAN RUANGAN 6
2.2.3 FITUR 3: APPROVAL, NOTIFIKASI & PELAPORAN 7
2.3 Pembagian Tugas Detail Anggota Kelompok 8
2.4 Class Diagram dan Struktur Kelas 13
2.5 Hubungan Antar Kelas dan Interaksi Sistem 17
2.6 Timeline Pengembangan Sistem 18
BAB III **22**
IMPLEMENTASI KONSEP OBJECT-ORIENTED PROGRAMMING **22**
3.1 Penerapan Prinsip OOP pada Sistem CLASSMATE 22
3.2 Encapsulation (Enkapsulasi) 22
3.3 Inheritance (Pewarisan) 22
3.4 Abstraction (Abstraksi) 22
3.5 Polymorphism (Polimorfisme) 23
3.6 Composition dan Aggregation 23
3.7 Interface dan Abstract Class dalam Desain Sistem 23
3.8 Penerapan Design Pattern (Pola Desain) 23
BAB IV **25**
KESIMPULAN **25**
4.1 Tujuan dan Solusi Utama 25
4.2 Implementasi Konsep OOP yang Kuat 25
4.3 Struktur dan Modularity Sistem 25
LAMPIRAN **27**
Tampilan Web Aplikasi Classmate 27
Hasil Pengujian SonarQube 33

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam lingkungan akademik, proses peminjaman ruang kelas sering kali dilakukan secara manual melalui pencatatan di buku atau spreadsheet. Hal ini dapat menyebabkan masalah seperti bentrok jadwal, data yang tidak konsisten, serta keterlambatan konfirmasi. Untuk mengatasi permasalahan tersebut, dibutuhkan sistem digital yang dapat mempermudah proses peminjaman ruang kelas secara efisien dan terstruktur.

Aplikasi CLASSMATE (Class Management Application Tool for Education) dirancang sebagai solusi berbasis Pemrograman Berorientasi Objek (PBO) yang membantu pengelolaan peminjaman ruang kelas di lingkungan kampus. Sistem ini dibangun menggunakan bahasa pemrograman Java dan dijalankan di Apache NetBeans dengan pendekatan OOP yang mencakup konsep inheritance, abstract class, dan interface.

1.2 Rumusan Masalah

1. Bagaimana merancang sistem yang dapat mengelola peminjaman ruang kelas secara digital?
2. Bagaimana menerapkan konsep OOP dalam perancangan dan implementasi sistem?
3. Bagaimana memastikan sistem dapat menghindari bentrok jadwal antar peminjam?

1.3 Batasan Masalah

1. Sistem hanya digunakan untuk peminjaman ruang kelas di lingkungan kampus.
2. Sistem berbasis desktop menggunakan Java dan NetBeans (tanpa koneksi web).
3. Autentikasi terbatas pada dua peran: **A**dmin dan Mahasiswa.
4. Penyimpanan data menggunakan database MySQL yang dihubungkan melalui JDBC Connector.

1.4 Tujuan

1. Membangun sistem peminjaman ruang kelas yang efisien, terstruktur, dan mudah digunakan.
2. Mengimplementasikan konsep dasar dan lanjutan dalam pemrograman berorientasi objek.
3. Mengembangkan keterampilan kolaborasi dan manajemen proyek tim dalam konteks pengembangan perangkat lunak.

1.5 Manfaat

1. Bagi Mahasiswa: Mempermudah peminjaman ruang kelas untuk kegiatan akademik.
2. Bagi Admin: Mempercepat proses verifikasi dan menghindari bentrok jadwal.
3. Bagi Pengembang: Sebagai sarana implementasi prinsip OOP secara nyata dalam proyek aplikasi.

BAB II

ANALISIS DAN PERANCANGAN SISTEM

2.1 Deskripsi Umum Sistem

Aplikasi CLASSMATE dirancang untuk mengelola proses peminjaman ruang kelas secara digital. Sistem ini memiliki dua jenis pengguna utama yaitu Admin dan Mahasiswa. Mahasiswa dapat melakukan peminjaman ruang kelas dengan menentukan waktu mulai dan waktu selesai peminjaman, sementara Admin berperan sebagai pihak yang memverifikasi permintaan tersebut dan mengatur ketersediaan ruang kelas.

Proses bisnis utama sistem ini meliputi:

1. Mahasiswa melakukan login dan membuat permintaan peminjaman ruang.
2. Sistem melakukan pengecekan ketersediaan ruang (menghindari jadwal bentrok).
3. Admin meninjau dan menyetujui/menolak permintaan peminjaman.
4. Sistem menyimpan data peminjaman dan memperbarui status ruang kelas.
5. Mahasiswa dapat melihat status dan riwayat peminjaman yang telah dilakukan.

Dengan pendekatan Object-Oriented Design (OOD), setiap entitas di sistem diwakili oleh class yang memiliki atribut dan method tersendiri. Prinsip encapsulation, inheritance, dan polymorphism diterapkan secara nyata dalam desain sistem ini.

2.2 Daftar Fitur Utama

2.2.1 FITUR 1: AUTENTIKASI & MANAJEMEN USER

Penanggung Jawab: Fazli Radika (103062300092)

Deskripsi Fitur:

Fitur ini bertanggung jawab untuk melakukan autentikasi pengguna dan pengelolaan data user dalam sistem CLASSMATE. Pengguna terdiri dari dua peran utama yaitu Admin dan Mahasiswa, di mana masing-masing memiliki hak akses yang berbeda. Admin memiliki akses penuh untuk memverifikasi peminjaman ruang dan mengelola data pengguna, sedangkan Mahasiswa hanya dapat melakukan peminjaman ruang dan melihat status peminjamannya.

Fungsionalitas:

1. Melakukan login dan logout berdasarkan data yang tersimpan dalam database.
2. Registrasi akun baru untuk Mahasiswa.
3. Mengatur role pengguna (Admin/Mahasiswa).
4. Menampilkan dan mengubah profil pengguna.
5. Menyimpan data pengguna secara terenkripsi di database MySQL.

Class yang Akan Dikembangkan:

1. User (Abstract Class)
    - Attributes (protected): int id, String nama, String email, String password, String role
    - Methods:
        - boolean login(String email, String password)
        - void logout()
        - abstract void viewProfile()
2. Admin (extends User)
    - Methods:
        - void manageUser(User user)
        - void approveBooking(Booking b)
3. Mahasiswa (extends User)
    - Attributes (private): String nim
    - Methods:
        - void requestBooking(BookingRequest req)
        - void viewBookings()

Justifikasi Desain:

1. Konsep inheritance digunakan untuk membedakan fungsi antara Admin dan Mahasiswa.
2. Abstract class User digunakan untuk menyatukan atribut dan method umum seluruh pengguna.
3. Encapsulation diterapkan dengan access modifier private/protected untuk menjaga keamanan data user.
4. Autentikasi menggunakan validasi login dengan koneksi JDBC ke database MySQL.

2.2.2 FITUR 2: BOOKING & PEMINJAMAN RUANGAN

Penanggung Jawab: Muhammad Afriza Hidayat (103062300093)

Deskripsi Fitur:

Fitur ini merupakan inti dari sistem CLASSMATE, yang berfungsi untuk memproses peminjaman ruang kelas. Mahasiswa dapat membuat permintaan peminjaman dengan menentukan ruang, tanggal, dan waktu. Sistem akan memeriksa ketersediaan ruang agar tidak terjadi bentrok jadwal, kemudian menyimpan data peminjaman yang valid ke dalam database.

Fungsionalitas:

1. Membuat peminjaman ruang berdasarkan waktu dan keperluan tertentu.
2. Mengecek konflik jadwal agar tidak terjadi double-booking.
3. Menyimpan data peminjaman ke tabel bookings.
4. Menampilkan riwayat peminjaman mahasiswa.
5. Membatalkan peminjaman sebelum jadwal dimulai.

Class yang Akan Dikembangkan:

1. Room
    - Attributes (private): int roomId, String roomName, int capacity, String location, boolean isAvailable
    - Methods:
        - boolean checkAvailability(LocalDateTime start, LocalDateTime end)
        - void updateStatus(boolean available)
2. Booking
    - Attributes (private): int bookingId, User requester, Room room, LocalDateTime startTime, LocalDateTime endTime, String purpose, String status
    - Methods:
        - boolean validateTime()
        - void saveBooking()
        - void cancelBooking()
3. BookingManager
    - Attributes: List<Booking> bookings
    - Methods:
        - Booking createBooking(BookingRequest request)
        - boolean checkConflict(Room room, LocalDateTime start, LocalDateTime end)
        - List<Booking> getUserBookings(int userId)

Justifikasi Desain:

1. Polymorphism diterapkan ketika objek Booking dikelola oleh BookingManager.

2. Encapsulation digunakan untuk mengamankan data waktu dan status peminjaman.
3. Abstraction melalui penggunaan class BookingRequest sebagai representasi input peminjaman.
4. Aggregation digunakan antara User dan Booking, karena satu pengguna dapat memiliki banyak booking.

2.2.3 FITUR 3: APPROVAL, NOTIFIKASI & PELAPORAN

Penanggung Jawab: Nomensen Melkisedek Pardosi (103062330101)

Deskripsi Fitur:

Fitur ini mencakup proses verifikasi peminjaman oleh Admin, pengiriman notifikasi hasil persetujuan, dan pembuatan laporan kegiatan peminjaman ruang. Setelah mahasiswa mengajukan peminjaman, Admin akan memutuskan apakah peminjaman disetujui atau ditolak. Hasil keputusan dikirimkan melalui notifikasi dan disimpan sebagai log dalam sistem.

Fungsionalitas:

1. Melihat daftar permintaan peminjaman yang menunggu approval.

2. Menyetujui atau menolak permintaan peminjaman.
3. Mengirimkan notifikasi status peminjaman ke peminjam.
4. Menampilkan laporan peminjaman per periode (harian/mingguan/bulanan).
5. Mengekspor laporan ke format CSV atau PDF.

Class yang Akan Dikembangkan:

1. ApprovalManager
    - Methods:
        - void approveBooking(Booking booking)
        - void rejectBooking(Booking booking, String reason)
2. Notification (Interface)
    - Method:
        - void send(User recipient, String message)
3. InAppNotification (implements Notification)
    - Method:
        - void send(User recipient, String message)
4. ReportGenerator
    - Methods:
        - void generateReport(LocalDate start, LocalDate end)
        - void exportToPDF()
        - void exportToCSV()

Justifikasi Desain:

1. Menggunakan interface Notification untuk fleksibilitas pengiriman pesan (bisa dikembangkan menjadi EmailNotification).

2. Polymorphism diterapkan antara kelas yang mengimplementasikan interface Notification.
3. Abstraction melalui ReportGenerator untuk memisahkan logika pelaporan dari proses bisnis utama.
4. Encapsulation menjaga keamanan data hasil laporan dan status persetujuan.

2.3 Pembagian Tugas Detail Anggota Kelompok

1. **Fazli Radika (103062300092)**
   Area Fokus: Autentikasi, Manajemen User, dan Struktur Database (Fondasi Sistem)

Tugas Utama	Kelas / Konsep OOP yang Diimplementasikan	Detail Tugas	Waktu (Estimasi Minggu)

I. Manajemen User & OOP	User (Abstract Class), Admin, Mahasiswa (Inheferitance)	Merancang dan mengimplementasikan konsep Inheritance antara User, Admin, dan Mahasiswa.	M2
		Menerapkan Encapsulation pada atribut User (password, email).	M2
II. Autentikasi & Database	DB Connection (Singleton), JDBC	Mengelola koneksi JDBC ke MySQL dan membuat skema tabel users.	M1
		Mengimplementasikan Singleton Pattern untuk kelas koneksi database (DBConnection).	M1
		Mengembangkan UserRepository (sesuai Repository Pattern).	M2
III. Desain Antarmuka	GUI Development (NetBeans)	Membuat antarmuka Login dan Registrasi Mahasiswa.	M3
		Membuat antarmuka Dashboard Admin (*Manage User*).	M4
IV. Pengujian	Unit Testing	Melakukan pengujian fungsionalitas Login/Logout dan Registrasi.	M4

2. **Muhammad Afriza Hidayat (103062300093)**
   Area Fokus: Booking, Logika Anti-Bentrok Jadwal, dan Logika Inti Sistem

Tugas Utama	Kelas / Konsep OOP yang Diimplementasikan	Detail Tugas	Waktu (Estimasi Minggu)
I. Logika Peminjaman Inti	Room, Booking, BookingManager (Aggregation, Abstraction)	Mengembangkan kelas Room dan Booking dengan penerapan Encapsulation.	M2
		Menerapkan Aggregation pada relasi Booking dengan Room dan User.	M2
		Membuat Abstract Class BookingBase (untuk abstraksi).	M2
II. Validasi & Konflik	BookingManager, Room.checkAvailability()	Mengimplementasikan logika paling krusial: checkConflict() untuk menghindari *double-booking*.	M2
		Mengimplementasikan logika pembatalan dan riwayat peminjaman (cancelBooking, viewBookings).	M3
III. Database & Akses Data	Repository Pattern	Merancang dan membuat skema tabel rooms dan bookings di MySQL.	M1
		Mengembangkan BookingRepository (sesuai Repository Pattern).	M2

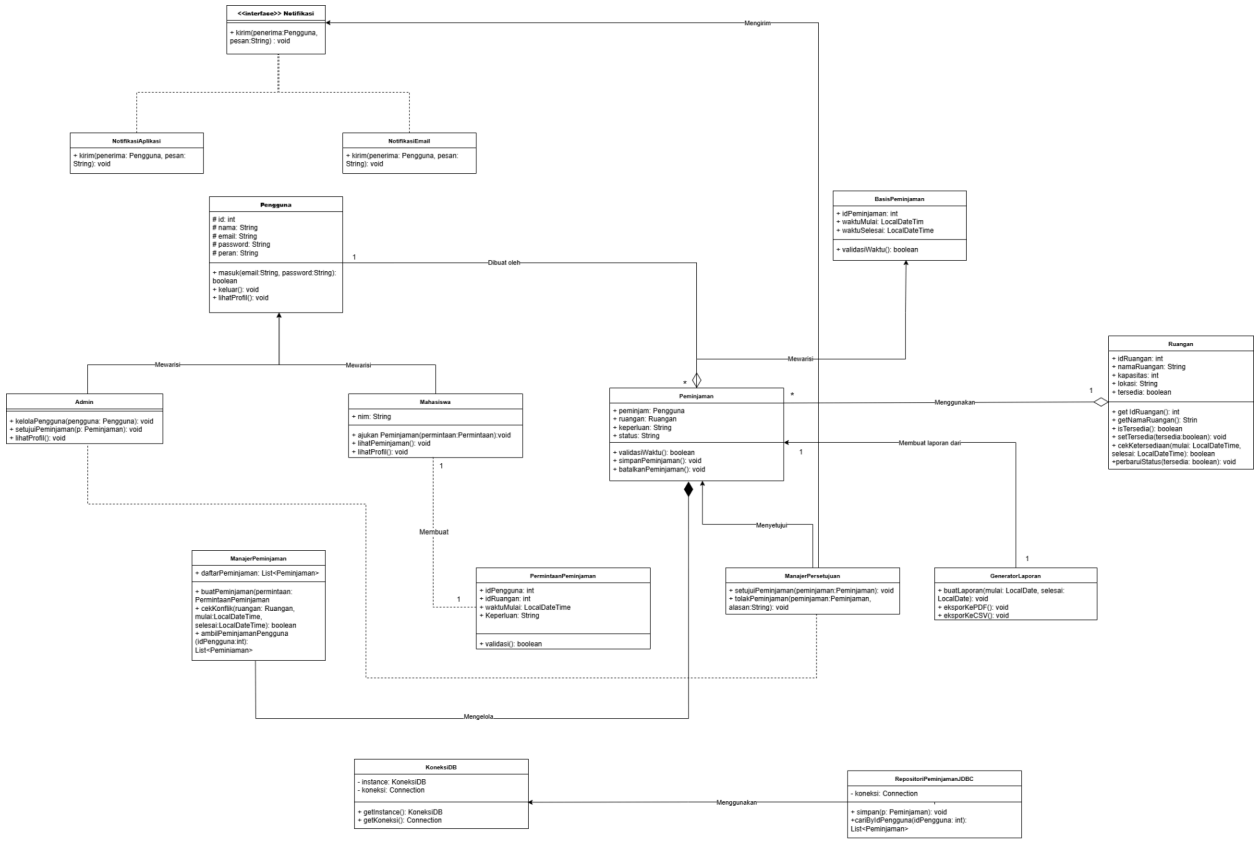
IV. Desain Antarmuka	GUI Development (NetBeans)	Membuat antarmuka Request Booking (Mahasiswa).	M3
		Membuat antarmuka Riwayat Peminjaman (Mahasiswa).	M3

3. **Nomensen Melkisedek Pardosi (103062330101)**
   Area Fokus: Approval, Notifikasi, Pelaporan, dan Implementasi Polimorfisme

Tugas Utama	Kelas / Konsep OOP yang Diimplementasikan	Detail Tugas	Waktu (Estimasi Minggu)
I. Notifikasi & Polimorfisme	Notification (Interface), InAppNotification (Polymorphism)	Merancang dan mengimplementasikan Interface Notification (kontrak untuk *send*).	M3
		Mengimplementasikan Polymorphism dengan membuat kelas InAppNotification yang mengimplementasikan interface tersebut.	M3
II. Approval Proses	ApprovalManager	Mengembangkan kelas ApprovalManager dan method approveBooking() dan rejectBooking().	M3
		Mengintegrasikan proses approval dengan sistem notifikasi.	M3

III. Pelaporan	ReportGenerator (Abstraction)	Membuat kelas ReportGenerator untuk abstraksi logika pelaporan.	M4
		Mengimplementasikan fungsionalitas generateReport (laporan per periode) dan exportToCSV().	M4
IV. Desain Antarmuka	GUI Development (NetBeans)	Membuat antarmuka Daftar Permintaan Approval (Admin).	M4
		Membuat antarmuka Laporan Peminjaman.	M4
V. Pengujian	System Testing	Memimpin pengujian sistem end-to-end, memastikan alur Mahasiswa Request → Admin Approve → Notifikasi berfungsi.	M4

2.4 Class Diagram dan Struktur Kelas



Class diagram berikut menggambarkan rancangan sistem CLASSMATE (Sistem Manajemen Peminjaman Ruang Kelas Berbasis Objek) yang dikembangkan menggunakan paradigma *Object-Oriented Programming* (OOP). Diagram ini menunjukkan hubungan antar kelas utama yang digunakan dalam sistem, meliputi entitas pengguna, peminjaman, ruang, manajer peminjaman, notifikasi, dan koneksi database.

1. Kelas Pengguna (Abstract Class)

   Kelas Pengguna berfungsi sebagai kelas dasar (superclass) bagi seluruh entitas pengguna sistem, seperti Admin dan Mahasiswa.

   Atribut yang dimiliki:

   - id (int)
   - nama (String)
   - email (String)
   - password (String)
   - peran (String)

   Method yang tersedia:

   - masuk(email, password): boolean → melakukan proses autentikasi pengguna.
   - keluar(): void → mengakhiri sesi pengguna.

- lihatProfil(): void → menampilkan informasi pengguna.

Kelas ini menerapkan prinsip abstraction dan inheritance, dimana method lihatProfil() bersifat umum dan diimplementasikan secara spesifik oleh kelas turunan.

2. Kelas Admin

Kelas Admin merupakan subclass dari Pengguna yang bertanggung jawab mengelola sistem.

Atribut khusus tidak ditambahkan, namun ia memiliki fungsionalitas tambahan:

- kelolaPengguna(pengguna: Pengguna): void → mengatur data pengguna.
- setujuiPeminjaman(peminjaman: Peminjaman): void → menyetujui atau menolak permintaan peminjaman ruang.

Kelas ini berperan penting dalam proses verifikasi dan validasi permintaan ruang. Dengan pewarisan dari Pengguna, kelas ini mewarisi atribut identitas dan fungsi login yang sama.

3. Kelas Mahasiswa

Kelas Mahasiswa juga merupakan subclass dari Pengguna.

Atribut tambahan:

- nim (String)

Method:

- ajukanPeminjaman(permintaan: PermintaanPeminjaman): void → membuat permintaan peminjaman ruang.
- lihatPeminjaman(): void → menampilkan daftar peminjaman yang pernah dilakukan.

Kelas ini merepresentasikan pengguna umum (end-user) yang dapat melakukan peminjaman ruang kelas.

4. Kelas Ruang

Kelas Kelas Ruang berfungsi untuk menyimpan data ruang yang dapat dipinjam.

Atribut:

- idRuang: int
- namaRuang: String
- kapasitas: int
- lokasi: String
- tersedia: boolean

Method:

- getIdRuang(): int
- getNamaRuang(): String

- isTersedia(): boolean
- setTersedia(status: boolean): void
- cekKetersediaan(waktuMulai, waktuSelesai): boolean
- perbaruiStatus(tersedia: boolean): void

Kelas ini menerapkan encapsulation, dengan atribut private yang hanya bisa diakses melalui getter/setter.

5. Kelas BasisPeminjaman (Abstract Class)

   Kelas abstrak ini menjadi dasar untuk proses peminjaman ruang.

   Atribut:

- idPeminjaman: int
- waktuMulai: LocalDateTime
- waktuSelesai: LocalDateTime

Method:

- validasiWaktu(): boolean → memastikan bahwa jadwal tidak bertabrakan.

Kelas ini menunjukkan penerapan abstraction, karena hanya mendefinisikan struktur umum, sementara implementasi detail dilakukan oleh subclass Peminjaman.

6. Kelas Peminjaman

   Kelas Peminjaman adalah turunan dari BasisPeminjaman yang mewakili transaksi peminjaman ruang kelas.

   Atribut:

- peminjam: Pengguna
- ruangan: Ruang
- keperluan: String
- status: String

Method:

- validasiWaktu(): boolean → memeriksa apakah ruang tersedia.
- simpanPeminjaman(): void → menyimpan data ke sistem.
- batalkanPeminjaman(): void → membatalkan peminjaman.

Kelas ini memiliki relasi composition dengan Ruang dan association dengan Pengguna.

7. Kelas PermintaanPeminjaman

Kelas ini menyimpan data pengajuan peminjaman dari mahasiswa sebelum disetujui oleh admin.

Atribut:

- idPengguna: int
- idRuang: int
- waktuMulai: LocalDateTime
- keperluan: String

Method:

- validasi(): boolean → memastikan kelengkapan data permintaan.

Objek PermintaanPeminjaman digunakan oleh ManajerPeminjaman untuk membuat objek Peminjaman baru.

8. Kelas ManajerPeminjaman

Kelas ManajerPeminjaman berfungsi mengelola seluruh proses peminjaman yang dilakukan oleh mahasiswa.

Atribut:

- daftarPeminjaman: List<Peminjaman>

Method:

- buatPeminjaman(permintaan: PermintaanPeminjaman): Peminjaman
- cekKonflik(ruang, waktuMulai, waktuSelesai): boolean
- ambilPeminjamanPengguna(idPengguna: int): List<Peminjaman>

Kelas ini mengatur logika bisnis inti sistem, seperti validasi dan penjadwalan ruang.

9. Kelas RepositoryPeminjaman (Interface)

Interface ini mendefinisikan kontrak untuk penyimpanan data peminjaman.

Method:

- simpan(peminjaman: Peminjaman): void
- cariByIdPengguna(idPengguna: int): List<Peminjaman>

Kelas ini diimplementasikan oleh kelas data konkret (misalnya dengan JDBC).

10. Kelas ManajerPersetujuan

Kelas ini khusus menangani proses persetujuan peminjaman oleh admin.

Method:

- setujuiPeminjaman(peminjaman: Peminjaman, alasan: String): void

- tolakPeminjaman(peminjaman: Peminjaman, alasan: String): void

Kelas ini memisahkan logika persetujuan dari pengelolaan peminjaman agar sistem lebih modular.

11. Kelas Notifikasi (Interface)

Interface ini mendefinisikan perilaku pengiriman pesan atau pemberitahuan ke pengguna.

Method:

- kirim(penerima: Pengguna, pesan: String): void

Dua kelas yang mengimplementasikannya:

- NotifikasiAplikasi
- NotifikasiEmail

Dengan pola ini, sistem dapat mengirim pesan ke berbagai media dengan polymorphism.

12. Kelas NotifikasiApplikasi dan NotifikasiEmail

Kedua kelas ini mengimplementasikan interface Notifikasi.

- NotifikasiAplikasi mengirimkan notifikasi melalui sistem aplikasi internal.
- NotifikasiEmail mengirimkan notifikasi melalui email.

Polymorphism memungkinkan sistem memilih jenis notifikasi secara dinamis tanpa mengubah logika inti.

13. Kelas GeneratorLaporan

Kelas ini berfungsi untuk membuat laporan kegiatan peminjaman ruang kelas.

Atribut:

- mulai: LocalDate
- selesai: LocalDate

Method:

- buatLaporan(): void
- eksporKePDF(): void
- eksporKeCSV(): void

Laporan diambil dari data Peminjaman yang dikelola ManajerPeminjaman.

2.5 Hubungan Antar Kelas dan Interaksi Sistem

Hubungan antar kelas pada sistem CLASSMATE menunjukkan bagaimana setiap komponen saling berinteraksi untuk membentuk alur peminjaman ruang yang terintegrasi dan berorientasi objek.

Kelas Pengguna menjadi induk dari kelas Admin dan Mahasiswa, yang menandakan adanya hubungan inheritance (pewarisan). Admin dan Mahasiswa mewarisi atribut serta method umum dari Pengguna, tetapi memiliki tanggung jawab yang berbeda sesuai perannya.

Kelas Mahasiswa berinteraksi dengan kelas ManajerPeminjaman melalui method ajukanPeminjaman() untuk membuat objek PermintaanPeminjaman, yang kemudian diproses menjadi Peminjaman jika data valid. Hubungan ini bersifat association, karena Mahasiswa menggunakan layanan dari ManajerPeminjaman.

Kelas Peminjaman memiliki hubungan composition dengan Ruang dan Pengguna, karena satu objek Peminjaman tidak dapat berdiri tanpa adanya ruang yang dipinjam dan pengguna yang meminjam.

Selanjutnya, Admin bekerja sama dengan ManajerPersetujuan untuk memverifikasi atau menolak permintaan yang masuk. Hasil keputusan tersebut dikirim kepada peminjam melalui interface Notifikasi, yang diimplementasikan oleh NotifikasiAplikasi dan NotifikasiEmail. Relasi ini mencerminkan polymorphism, karena sistem dapat mengirim notifikasi menggunakan berbagai media tanpa mengubah kode utama.

Kelas ManajerPeminjaman menyimpan daftar seluruh data Peminjaman dan memiliki hubungan aggregation, karena daftar tersebut dapat berubah-ubah selama sistem berjalan. Data peminjaman disimpan melalui RepositoryPeminjaman, sebuah interface yang berfungsi sebagai lapisan penyimpanan data (repository pattern).

Kelas GeneratorLaporan memiliki hubungan association dengan ManajerPeminjaman, karena laporan yang dihasilkan diambil dari daftar data peminjaman yang dikelola.

Sementara itu, kelas BasisPeminjaman bertindak sebagai kelas abstrak yang diwarisi oleh Peminjaman, menyediakan kerangka atribut dan method dasar seperti validasi waktu.

Secara keseluruhan, relasi antar kelas pada sistem ini membentuk pola hierarki dan modular: pengguna berinteraksi dengan manajer, manajer mengelola transaksi, dan hasil transaksi diteruskan ke lapisan notifikasi serta laporan. Desain ini memperlihatkan penerapan prinsip encapsulation, inheritance, abstraction, dan polymorphism secara nyata dalam sistem manajemen peminjaman ruang kelas berbasis objek.

2.6 Timeline Pengembangan Sistem

Tahap	Minggu	Durasi (Hari Kerja)	Aktivitas Utama & Deliverables	Penanggung Jawab Utama
Tahap I: Perancangan & Database	Minggu 1	5 Hari	M1.1 Database & Struktur Awal	Tim
			Finalisasi Class Diagram (Relasi Aggregation/Composition).	Tim
			Instalasi MySQL & JDBC Connector.	Tim
			Perancangan dan Implementasi Skema Database (users, rooms, bookings, dll.).	Fazli Radika
			Pembuatan Interface DBConnection dan implementasi Singleton Pattern.	Fazli Radika
Tahap II: Implementasi Inti (Fitur 1 & 2)	Minggu 2	5 Hari	M2.1 Implementasi Kelas User & Autentikasi	Fazli Radika
			Coding: Abstract Class User, Admin, Mahasiswa (Inheritance).	Fazli Radika
			Implementasi Login/Logout (termasuk enkapsulasi password).	Fazli Radika

			M2.2 Implementasi Kelas Room & Booking	M. Afriza Hidayat
			Coding: Kelas Room dan Booking.	M. Afriza Hidayat
			Implementasi BookingManager.checkConflict() (Logika anti-bentrok jadwal).	M. Afriza Hidayat
Tahap III: Implementasi Fungsionalitas Lanjutan (Fitur 3 & UI)	Minggu 3	5 Hari	M3.1 Implementasi Approval & Notifikasi	Nomensen P.
			Coding: Interface Notification & InAppNotification (Polymorphism).	Nomensen P.
			Coding: ApprovalManager (Logika persetujuan/penolakan peminjaman).	Nomensen P.
			M3.2 Pembuatan Antarmuka Pengguna (GUI)	Tim
			Desain dan Implementasi Form Login & Registrasi (NetBeans Swing/JavaFX).	Fazli Radika
			Desain Dashboard Mahasiswa (Form *Request Booking* & *View Bookings*).	M. Afriza Hidayat

Tahap IV: Pengujian, Laporan & Finalisasi	Minggu 4	5 Hari	M4.1 Pengujian Sistem (Unit Testing)	Tim
			Testing Fungsionalitas Inti: Login, Validasi Waktu, Approval (Negatif & Positif Test).	M. Afriza Hidayat & Nomensen P.
			M4.2 Implementasi Fitur Pelaporan	Nomensen P.
			Coding: ReportGenerator (Menampilkan laporan historis).	Nomensen P.
			M4.3 Finalisasi GUI & Integrasi	Tim
			Desain Dashboard Admin (Form *Approval List* & *Manage User*).	Fazli Radika
			Integrasi semua modul dan GUI.	Tim
Tambahan	Minggu 5	5 Hari	M5.1 Dokumentasi & *Deployment*	Tim
			Finalisasi laporan, dokumentasi kode (Javadoc), dan *user manual*.	Tim
			Persiapan *executable* (.jar) untuk presentasi.	Tim

BAB III

IMPLEMENTASI KONSEP OBJECT-ORIENTED PROGRAMMING

3.1 Penerapan Prinsip OOP pada Sistem CLASSMATE

Sistem CLASSMATE dibangun dengan menerapkan seluruh prinsip utama dalam Pemrograman Berorientasi Objek (Object-Oriented Programming), yaitu *Encapsulation*, *Inheritance*, *Abstraction*, dan *Polymorphism*. Keempat konsep ini diimplementasikan secara terstruktur agar aplikasi tidak hanya berfungsi dengan baik, tetapi juga mudah dikembangkan, dipelihara, dan diuji di masa depan. Setiap konsep memiliki peran penting dalam menjaga modularitas kode dan mencerminkan objek nyata ke dalam bentuk kelas dan relasi antarkelas di sistem.

3.2 Encapsulation (Enkapsulasi)

Konsep enkapsulasi diterapkan untuk menjaga keamanan dan integritas data pada setiap kelas dalam sistem. Atribut pada kelas seperti Room, Booking, dan User disembunyikan (menggunakan access modifier private atau protected) agar tidak dapat diakses langsung dari luar kelas. Pengaksesan dan modifikasi nilai atribut dilakukan melalui *getter* dan *setter* yang memiliki validasi internal. Contohnya, pada kelas Room, atribut available hanya dapat diubah menggunakan method setAvailable() agar perubahan status ruangan tetap terkendali. Dengan cara ini, sistem mampu mencegah manipulasi data secara langsung dan memastikan setiap data yang masuk tetap valid.

3.3 Inheritance (Pewarisan)

Inheritance digunakan untuk mewariskan atribut dan perilaku umum ke kelas turunan. Dalam sistem CLASSMATE, kelas User dijadikan sebagai *superclass* yang memiliki atribut dan method dasar seperti id, name, email, dan login(). Kelas Admin dan Mahasiswa menjadi *subclass* yang mewarisi atribut dan method dari User, tetapi juga memiliki perilaku tambahan sesuai dengan peran masing-masing. Contohnya, Admin memiliki kemampuan untuk memverifikasi peminjaman ruang, sedangkan Mahasiswa dapat membuat permintaan peminjaman. Dengan pewarisan ini, kode menjadi lebih efisien dan mudah diperluas tanpa perlu menulis ulang logika yang sama untuk setiap tipe pengguna.

3.4 Abstraction (Abstraksi)

Konsep abstraksi diwujudkan melalui penggunaan abstract class dan interface yang berfungsi untuk menyembunyikan kompleksitas implementasi dari pengguna kelas lain. Sebagai contoh, kelas BookingBase dijadikan *abstract class* yang menjadi blueprint untuk proses peminjaman ruang. Kelas ini mendefinisikan method validateTime() yang wajib

diimplementasikan oleh kelas turunannya, Booking. Sementara itu, interface Notification digunakan untuk mendefinisikan kontrak pengiriman notifikasi tanpa harus mengetahui bagaimana cara notifikasi tersebut dikirim (misalnya, melalui aplikasi atau email). Pendekatan ini memastikan bahwa sistem dapat diperluas dengan mudah di masa depan, misalnya dengan menambahkan jenis notifikasi baru tanpa mengubah struktur utama aplikasi.

3.5 Polymorphism (Polimorfisme)

Polimorfisme memungkinkan satu referensi objek untuk merujuk ke berbagai bentuk kelas turunan. Implementasinya pada sistem CLASSMATE terlihat pada penggunaan interface Notification. Sebagai contoh, variabel bertipe Notification dapat diisi dengan objek InAppNotification atau EmailNotification tanpa harus mengubah logika pengiriman notifikasi di dalam kode. Hal ini membuat sistem menjadi lebih fleksibel dan adaptif terhadap perubahan kebutuhan. Polimorfisme juga diterapkan pada proses approval, di mana Admin dapat memanggil method approveBooking() yang bersifat generik, tetapi hasil implementasinya dapat berbeda sesuai tipe objek yang diproses.

3.6 Composition dan Aggregation

Sistem CLASSMATE menggunakan composition dan aggregation untuk membangun hubungan antarobjek. Kelas Booking memiliki relasi dengan kelas Room dan User, di mana setiap peminjaman harus dikaitkan dengan satu pengguna dan satu ruang tertentu. Hubungan ini bersifat *aggregation*, karena Booking tidak memiliki *lifecycle* yang bergantung pada Room atau User. Artinya, ketika objek Booking dihapus, objek Room tetap ada. Sementara itu, composition digunakan dalam kelas seperti BookingManager yang mengelola daftar peminjaman melalui koleksi objek Booking. Desain ini memastikan sistem dapat merepresentasikan hubungan nyata antar entitas dengan cara yang terstruktur.

3.7 Interface dan Abstract Class dalam Desain Sistem

Penggunaan interface dan abstract class pada sistem CLASSMATE menjadi fondasi penting dalam membangun arsitektur yang modular. Interface Notification memberikan fleksibilitas bagi pengembang untuk menambahkan berbagai tipe notifikasi di masa mendatang tanpa perlu memodifikasi logika utama sistem. Sementara itu, abstract class BookingBase digunakan untuk menyediakan struktur dasar bagi semua proses peminjaman ruang, seperti validasi waktu dan status peminjaman. Kedua pendekatan ini mencerminkan penerapan prinsip *open/closed principle*, yaitu sistem terbuka untuk dikembangkan tetapi tertutup untuk modifikasi yang berlebihan.

3.8 Penerapan Design Pattern (Pola Desain)

Sistem CLASSMATE menggunakan dua pola desain penting:

1. Singleton Pattern (DBConnection), digunakan untuk memastikan hanya ada satu koneksi database aktif selama runtime.
2. Repository Pattern (BookingRepository), memisahkan logika bisnis dari akses database agar sistem lebih modular dan mudah diuji.

Dengan adanya kelas seperti Booking Repository JDBC dan User Repository, sistem dapat dengan mudah mengganti atau menambah sumber data tanpa mempengaruhi bagian lain dari program. Penerapan design pattern ini tidak hanya membuat kode lebih rapi, tetapi juga memperkuat penerapan prinsip OOP dalam proyek ini.

BAB IV

KESIMPULAN

Proyek Aplikasi CLASSMATE adalah solusi digital berbasis Pemrograman Berorientasi Objek (PBO) menggunakan Java dan NetBeans dengan database MySQL untuk mengatasi inefisiensi dan bentrok jadwal dalam proses peminjaman ruang kelas di lingkungan akademik.

4.1 Tujuan dan Solusi Utama

Tujuan utama proyek ini adalah membangun sistem peminjaman ruang kelas yang efisien, terstruktur, dan mudah digunakan, yang secara spesifik dirancang untuk menghindari bentrok jadwal melalui pengecekan ketersediaan ruang secara otomatis. Sistem ini membagi pengguna menjadi dua peran utama, Admin dan Mahasiswa, dengan hak akses yang jelas dan berbeda.

4.2 Implementasi Konsep OOP yang Kuat

Kesuksesan perancangan sistem ini sangat bergantung pada implementasi menyeluruh dari prinsip-prinsip PBO:

1. Inheritance (Pewarisan): Diterapkan pada kelas User (Abstract Class), di mana kelas Admin dan Mahasiswa mewarisi properti umum, namun memiliki metode fungsionalitas yang spesifik.
2. Encapsulation (Enkapsulasi): Digunakan untuk menjaga integritas dan keamanan data melalui pembatasan akses (private/protected) pada atribut di kelas-kelas inti seperti User, Room, dan Booking.
3. Abstraction (Abstraksi): Diwujudkan melalui penggunaan Abstract Class (BookingBase) untuk mendefinisikan blueprint fungsionalitas dan Interface (Notification) untuk menyembunyikan detail implementasi, memastikan modularitas.
4. Polymorphism (Polimorfisme): Diterapkan pada objek yang mengimplementasikan Interface Notification dan pada proses approval yang memungkinkan referensi tunggal merujuk pada berbagai bentuk objek turunan.

4.3 Struktur dan Modularity Sistem

Sistem dibagi menjadi tiga fitur utama yang dikelola oleh anggota tim yang berbeda, yang menunjukkan manajemen proyek yang terstruktur:

1. Autentikasi & Manajemen User (Kelas User, Admin, Mahasiswa).
2. Booking & Peminjaman Ruangan (Kelas Room, Booking, BookingManager).
3. Approval, Notifikasi & Pelaporan (Kelas ApprovalManager, Interface Notification, ReportGenerator).

Selain itu, modularitas diperkuat dengan penggunaan Design Pattern seperti Singleton Pattern untuk koneksi database (DBConnection) dan Repository Pattern untuk memisahkan logika akses data, yang menjadikan sistem mudah dikembangkan, dipelihara, dan diuji (Open/Closed Principle).

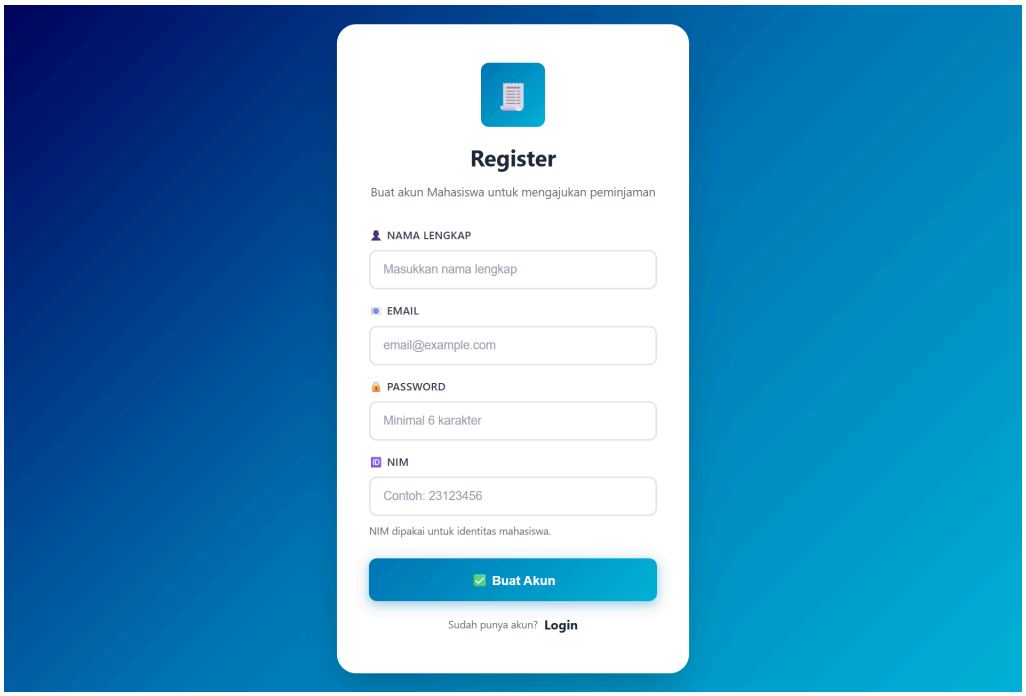
Secara keseluruhan, Aplikasi CLASSMATE tidak hanya memenuhi kebutuhan fungsional untuk digitalisasi peminjaman ruang kelas, tetapi juga menjadi sarana implementasi konsep-konsep PBO secara nyata, menghasilkan kode yang efisien, terstruktur, dan siap dikembangkan lebih lanjut.

LAMPIRAN

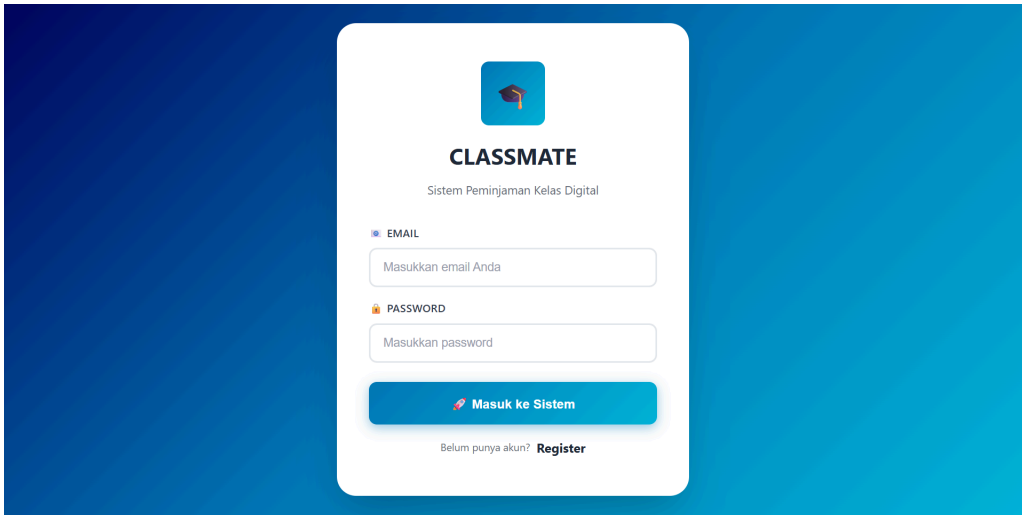
Tampilan Web Aplikasi Classmate

Role Mahasiswa

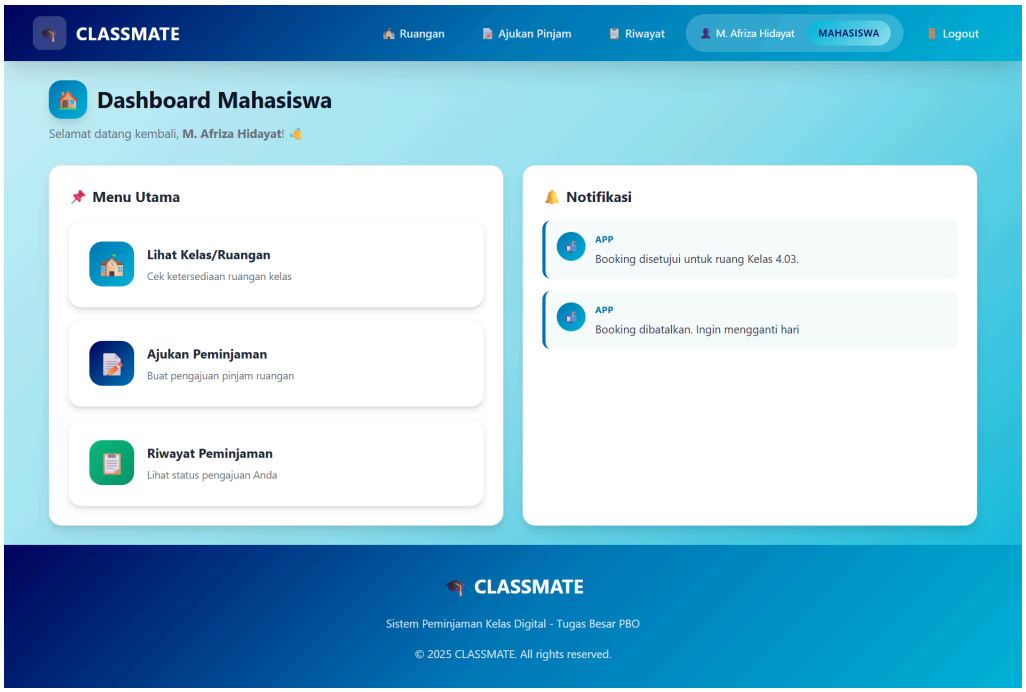
1. Halaman Register/Daftar



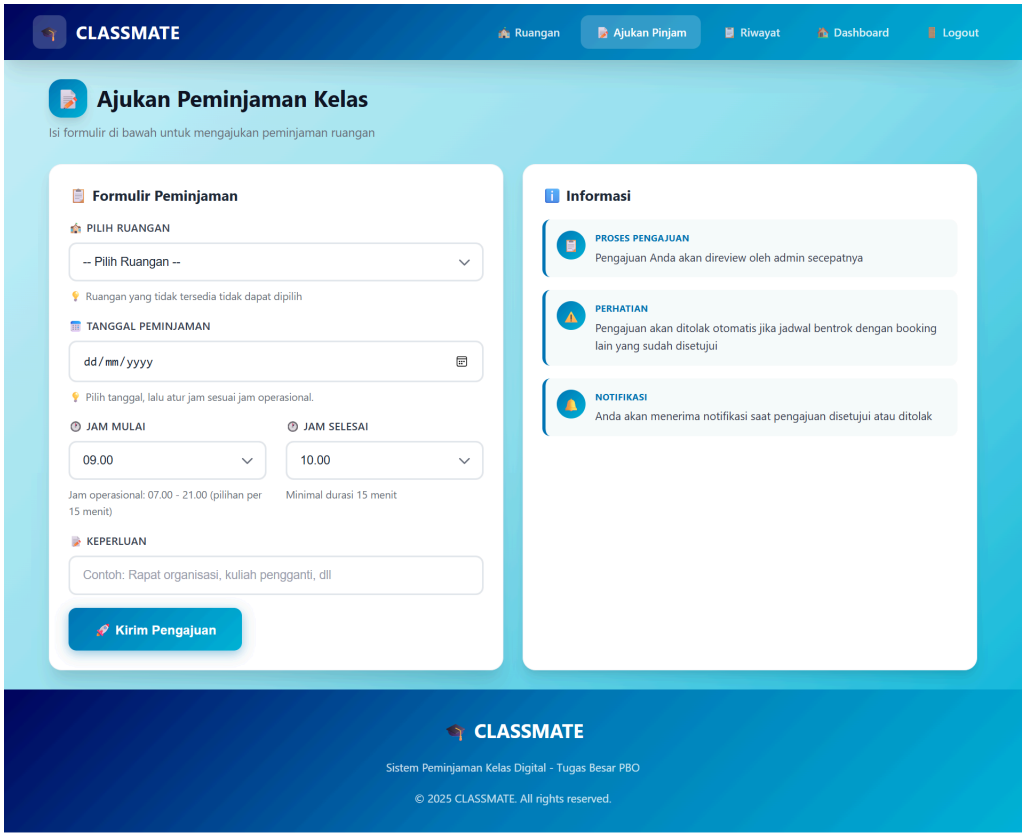
2. Halaman Login



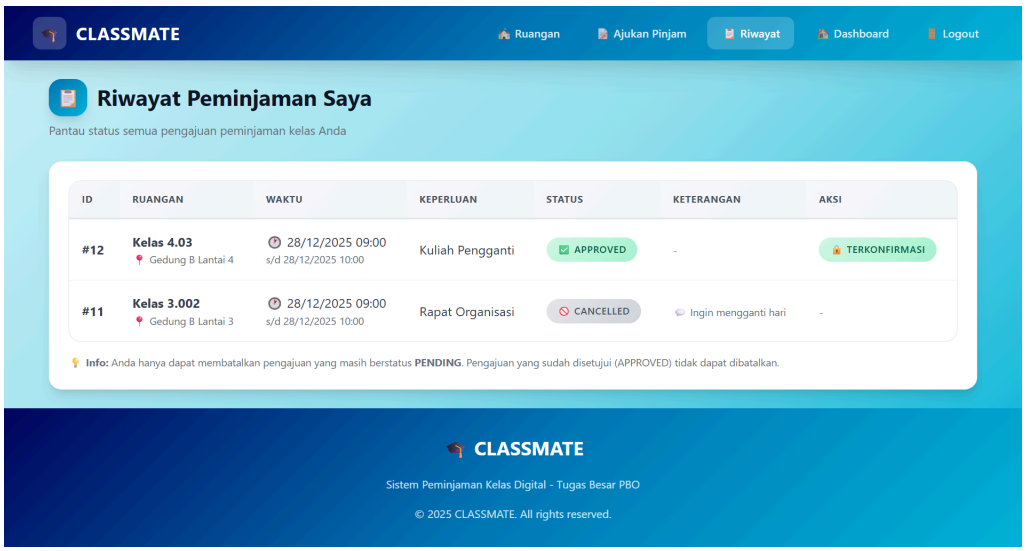
3. Halaman Dashboard Mahasiswa



4. Halaman Daftar Kelas/Ruangan

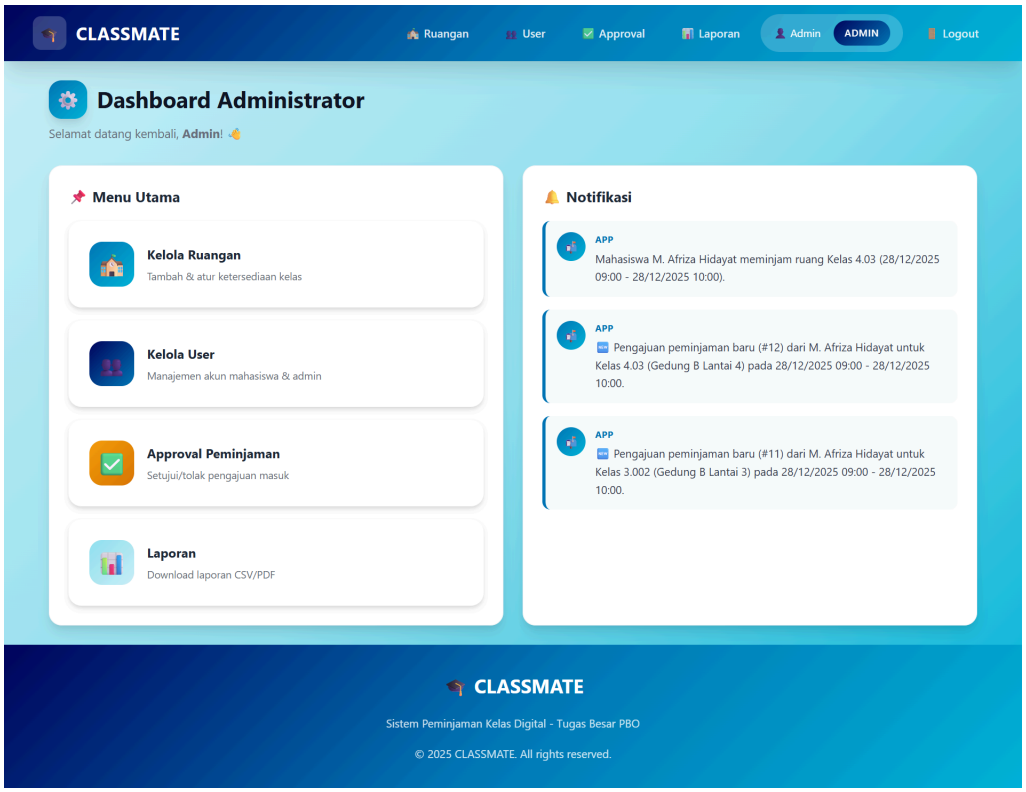


5. Halaman Riwayat Peminjaman Kelas

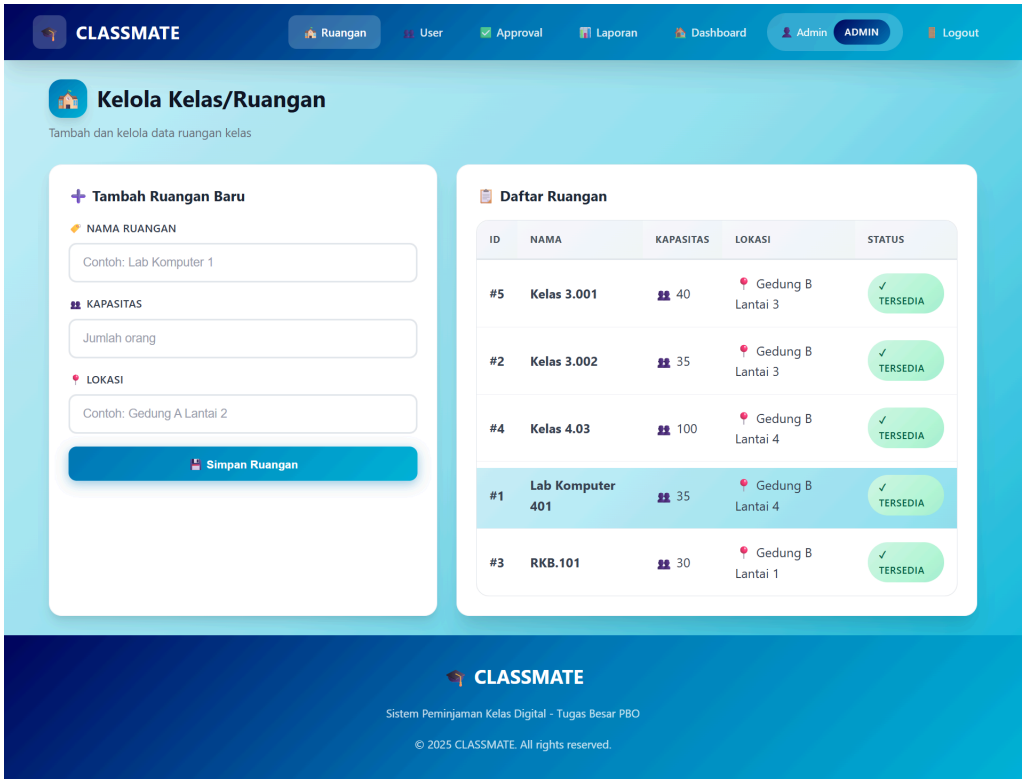


Role Admin

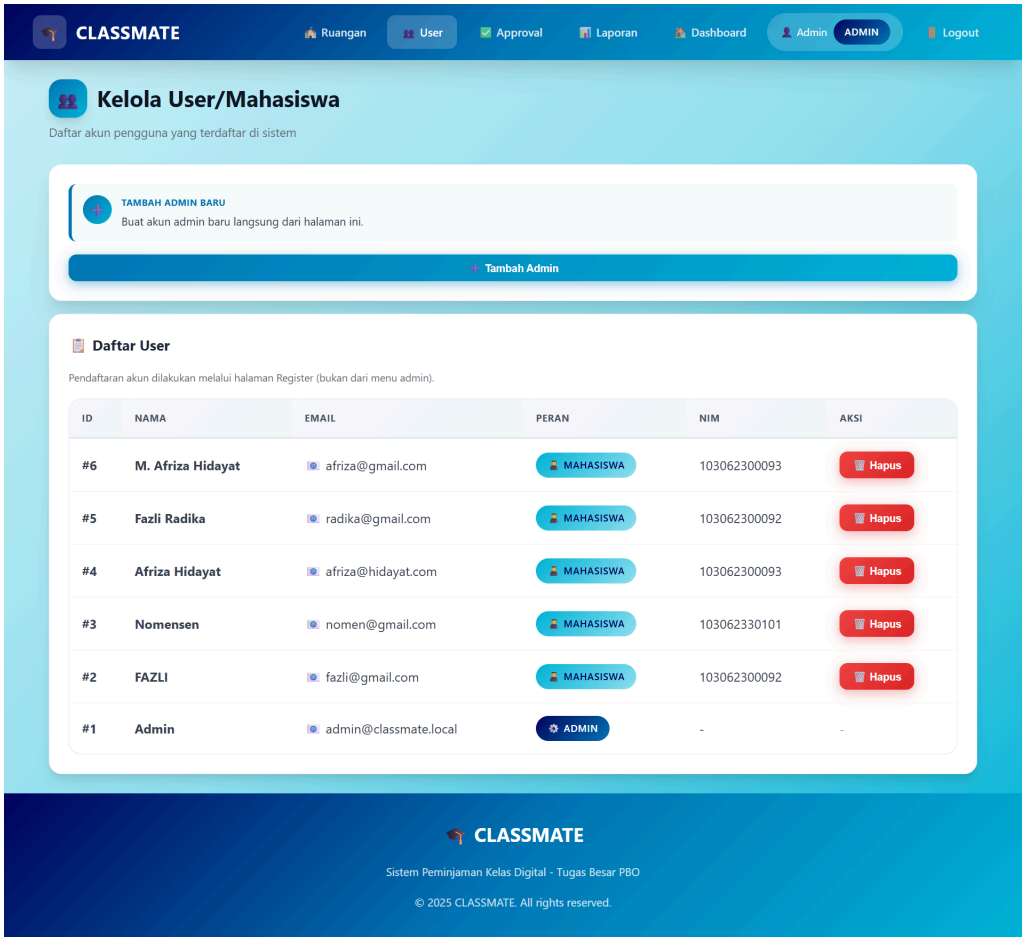
6. Halaman Dashboard Admin



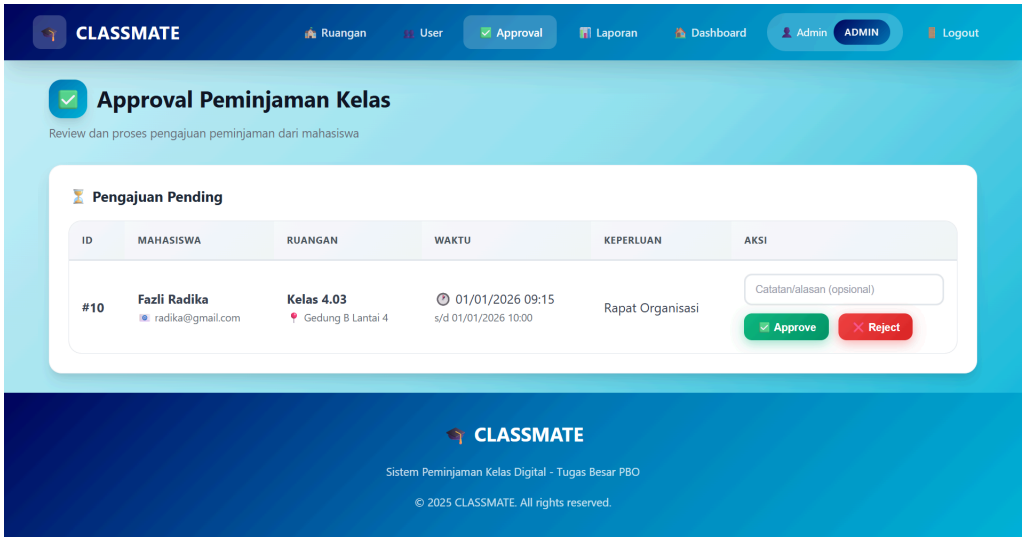
7. Halaman Kelola Ruangan



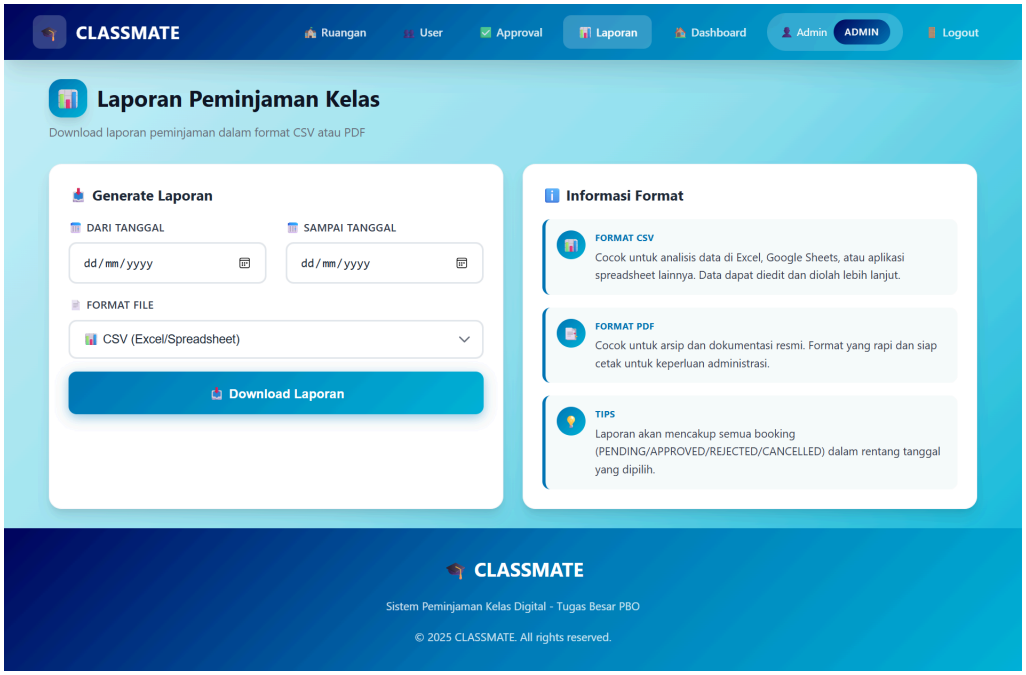
8. Halaman Kelola User



9. Halaman Persetujuan/Approval

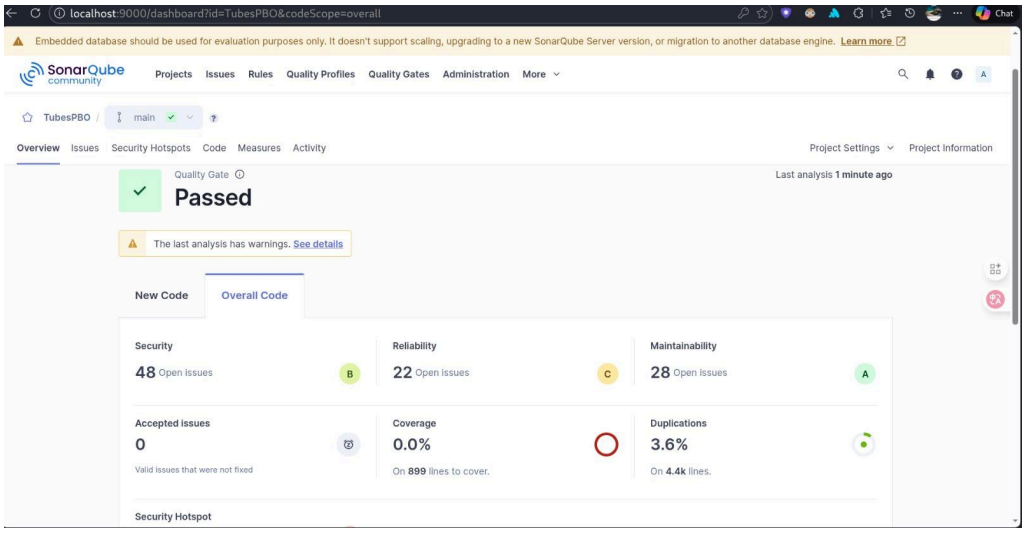


10. Halaman Laporan Peminjaman Kelas



Hasil Pengujian SonarQube

1. Kualitas kode proyek “CLASSMATE” di SonarQube.



2. Project Summary



Berdasarkan hasil analisis pada SonarQube, proyek CLASSMATE dinyatakan lulus Quality Gate (Passed), yang berarti proyek telah memenuhi standar kualitas minimum yang ditetapkan. Meskipun demikian, masih terdapat beberapa peringatan dari hasil analisis. Pada aspek keamanan (Security) ditemukan 48 issue dengan nilai B, yang menunjukkan adanya potensi kerentanan namun belum bersifat kritis. Pada aspek keandalan (Reliability) terdapat 22 issue dengan nilai C, yang mengindikasikan kemungkinan bug atau kesalahan logika dalam kode program. Sementara itu, aspek maintainability memperoleh nilai A dengan 28 issue, menandakan bahwa struktur kode relatif baik dan mudah untuk dipelihara.

Selain itu, hasil analisis menunjukkan coverage sebesar 0.0%, yang berarti belum terdapat pengujian unit (unit testing) yang terdeteksi dalam proyek. Tingkat duplikasi kode sebesar 3.6% masih berada dalam batas yang wajar dan tidak berdampak signifikan terhadap kualitas keseluruhan. Proyek ini memiliki sekitar 3.8 ribu baris kode yang ditulis menggunakan bahasa Java dan JSP, serta bersifat public. Meskipun proyek telah lolos standar kualitas, disarankan untuk meningkatkan kualitas lebih lanjut dengan menambahkan pengujian unit serta meninjau dan memperbaiki issue pada aspek keamanan dan keandalan agar kualitas perangkat lunak menjadi lebih optimal.